

NiceGUI 中 `ui.run_with(app)` 的全维度解析

`ui.run_with(app)` 是 NiceGUI 提供的**自定义应用启动入口** API，用于将 NiceGUI 集成到已有的 FastAPI/Starlette 应用中，而非使用 NiceGUI 默认的独立应用实例。它突破了 `ui.run()` 的封装限制，支持与现有 Web 服务共享路由、中间件、依赖注入等核心能力，是实现“NiceGUI 作为子模块嵌入现有 Web 应用”的关键工具。

一、核心作用与适用场景

1. 核心作用

- 复用已有 FastAPI/Starlette 应用实例，将 NiceGUI 的 UI 组件 / 路由挂载到该应用上；
- 共享现有应用的中间件（认证、日志、跨域）、依赖项、配置（端口、域名、HTTPS）；
- 实现 NiceGUI UI 与原生 FastAPI 接口的无缝集成（如同一端口同时提供 API 和可视化界面）；
- 自定义应用启动逻辑（如先初始化数据库，再启动 Web 服务）。

2. 典型适用场景

场景	示例
集成到现有 FastAPI 服务	已有 FastAPI 接口服务，新增 NiceGUI 可视化仪表盘
共享中间件 / 认证	复用 JWT 认证中间件，控制 NiceGUI 页面访问权限
自定义启动流程	先初始化 Redis / 数据库连接，再启动 Web 服务
多路由前缀隔离	将 NiceGUI 挂载到 <code>/ui</code> 路径，API 挂载到 <code>/api</code> 路径
生产环境部署	结合 Gunicorn/Uvicorn 部署，复用生产级配置

二、基本语法与使用方式

1. 核心前置知识

NiceGUI 底层基于 FastAPI (Starlette) 构建，`ui.run()` 本质是创建一个默认 FastAPI 应用并启动；`ui.run_with(app)` 则是将 NiceGUI 的路由 / 组件注册到**用户自定义的 FastAPI 应用实例**上，核心流程：

- 创建自定义 FastAPI 应用实例；
- 配置该应用（中间件、依赖、路由等）；
- 调用 `ui.run_with(app)` 将 NiceGUI 集成到该应用；
- 启动应用（可使用 Uvicorn/Gunicorn，而非 NiceGUI 默认的启动器）。

2. 基础语法

```
from nicegui import ui
from fastapi import FastAPI
import uvicorn

# 1. 创建自定义FastAPI应用
app = FastAPI(title='自定义应用', version='1.0')

# 2. 配置自定义应用（可选：中间件、API路由、依赖等）
@app.get('/api/hello')
async def hello_api():
    return {'message': 'Hello from FastAPI'}

# 3. 定义NiceGUI UI组件
ui.label('集成到自定义FastAPI应用的NiceGUI界面').classes('text-2xl')
ui.button('调用API', on_click=lambda: ui.notify('调用/api/hello成功'))

# 4. 用ui.run_with挂载NiceGUI到自定义应用
# 方式1: 使用NiceGUI内置启动器（简单，兼容ui.run参数）
ui.run_with(app, port=8080, reload=True)

# 方式2: 手动启动（推荐生产环境，更灵活）
# if __name__ == '__main__':
#     ui.run_with(app) # 仅挂载，不启动
#     uvicorn.run(app, host='0.0.0.0', port=8080)
```

3. 核心参数详解

`ui.run_with(app, **kwargs)` 的参数与 `ui.run()` 完全兼容，核心参数：

参数名	类型	取值说明	默认值
<code>app</code>	FastAPI/Starlette app	必选，自定义的 FastAPI/Starlette 应用实例	无
<code>port</code>	int	服务端口号	8080
<code>host</code>	str	绑定地址（0.0.0.0 允许外网访问）	'127.0.0.1'
<code>reload</code>	bool	开发模式热重载（修改代码自动重启）	False
<code>title</code>	str	浏览器标签页标题	'NiceGUI'
<code>favicon</code>	str	页面图标路径（URL / 本地路径）	None
<code>uvicorn_logging_level</code>	str	Uvicorn 日志级别（debug/info/warning/error）	'warning'
<code>mount_path</code>	str	NiceGUI 挂载的 URL 前缀（如 <code>/ui</code> ，则 NiceGUI 界面在 <code>http://host:port/ui</code> ）	'/'

4. 场景 1：集成到现有 FastAPI 服务（核心场景）

已有 FastAPI API 服务，新增 NiceGUI 可视化界面，共享端口和认证中间件：

```
from nicegui import ui
from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

# 1. 创建现有FastAPI应用
app = FastAPI(title='业务系统API')

# 2. 配置认证中间件（示例：简单Token认证）
security = HTTPBearer()
def verify_token(credentials: HTTPAuthorizationCredentials = Depends(security)):
    if credentials.credentials != 'valid-token':
        raise HTTPException(status_code=401, detail='无效Token')
    return credentials

# 3. 现有API路由（带认证）
@app.get('/api/data', dependencies=[Depends(verify_token)])
async def get_data():
    return {'data': [1, 2, 3], 'total': 3}

# 4. 定义NiceGUI UI（挂载到/ui路径，复用认证）
ui.run_with(app, mount_path='/ui') # 先挂载，再定义UI

# NiceGUI页面（调用带认证的API）
async def fetch_data():
    try:
        # 调用同应用的API，携带Token
        response = await ui.request.get('/api/data', headers={'Authorization': 'Bearer valid-token'})
        ui.notify(f'API返回: {response.json()}')
    except Exception as e:
        ui.notify(f'调用失败: {e}', type='error')

ui.label('业务系统可视化界面').classes('text-2xl my-5')
ui.button('获取API数据', on_click=fetch_data)

# 5. 手动启动应用（生产环境推荐）
if __name__ == '__main__':
    import uvicorn
    uvicorn.run(app, host='0.0.0.0', port=8080)
```

访问说明：

- API 地址： `http://localhost:8080/api/data`（需带 Token）；
- NiceGUI 界面： `http://localhost:8080/ui`（可直接访问，内部调用 API 时携带 Token）。

5. 场景 2：自定义启动流程（先初始化资源）

启动 Web 服务前先初始化数据库、Redis 等资源，再挂载 NiceGUI:

```
from nicegui import ui
from fastapi import FastAPI
import uvicorn
import redis

# 1. 初始化资源（启动前执行）
def init_resources():
    # 初始化Redis连接
    global redis_client
    redis_client = redis.Redis(host='localhost', port=6379, db=0)
    print('Redis初始化完成')

# 2. 创建FastAPI应用
app = FastAPI()

# 3. 挂载NiceGUI前先初始化资源
init_resources()

# 4. 挂载NiceGUI到应用
ui.run_with(app, mount_path='/dashboard')

# NiceGUI页面使用Redis资源
def get_redis_data():
    count = redis_client.get('page_views') or 0
    redis_client.set('page_views', int(count) + 1)
    ui.label(f'页面访问次数: {int(count) + 1}').classes('text-xl')

ui.label('数据仪表盘（带Redis统计）').classes('text-2xl my-5')
ui.button('刷新访问次数', on_click=get_redis_data)

# 5. 启动应用
if __name__ == '__main__':
    uvicorn.run(app, host='0.0.0.0', port=8080)
```

6. 场景 3：多应用隔离（挂载到不同前缀）

将多个 NiceGUI 模块挂载到不同 URL 前缀，或其他 Starlette 应用集成:

```
from nicegui import ui
from fastapi import FastAPI
from starlette.mount import Mount

# 1. 创建根应用
root_app = FastAPI()

# 2. 创建子应用1（用户管理UI）
app_user = FastAPI()
ui.run_with(app_user, mount_path='/', title='用户管理') # 子应用内挂载到根
```

```
ui.label('用户管理界面').classes('text-2x1')

# 3. 创建子应用2（订单管理UI）
app_order = FastAPI()
ui.run_with(app_order, mount_path='/', title='订单管理')
ui.label('订单管理界面').classes('text-2x1')

# 4. 将子应用挂载到根应用的不同前缀
root_app.mount('/user', app_user)
root_app.mount('/order', app_order)

# 5. 根应用的API路由
@root_app.get('/api/root')
async def root_api():
    return {'message': '根API'}

# 6. 启动根应用
if __name__ == '__main__':
    import uvicorn
    uvicorn.run(root_app, host='0.0.0.0', port=8080)
```

访问说明：

- 用户管理 UI: `http://localhost:8080/user`；
- 订单管理 UI: `http://localhost:8080/order`；
- 根 API: `http://localhost:8080/api/root`。

三、关键特性与注意事项

1. 与 `ui.run()` 的核心区别

特性	<code>ui.run_with(app)</code>	<code>ui.run()</code>
应用实例	使用用户自定义 FastAPI/Starlette 实例	创建默认 FastAPI 实例
启动方式	可手动用 Uvicorn/Gunicorn 启动	内置启动器（封装 Uvicorn）
灵活性	高（支持全量 FastAPI 配置）	低（仅支持有限参数）
集成场景	嵌入现有 Web 应用	独立运行 NiceGUI 应用
中间件 / 依赖	共享自定义配置	使用 NiceGUI 默认配置

2. 挂载路径（`mount_path`）注意事项

- `mount_path` 是 NiceGUI 在自定义应用中的 URL 前缀，默认 `/`（根路径）；
- 若设置 `mount_path='/ui'`，则所有 NiceGUI 组件 / 路由都在 `/ui` 下（如 `/ui` 是主界面，`/ui/_nicegui` 是内置静态资源）；
- 避免与现有 API 路由冲突（如 API 有 `/ui` 路由，则 NiceGUI 需换前缀）。

3. 热重载 (reload) 配置

- 开发模式下设置 `reload=True` 可实现代码热更新，但需注意：
 - `ui.run_with(app, reload=True)` 会使用 NiceGUI 内置的热重载，仅监控 NiceGUI 相关代码；
 - 若需监控 FastAPI 代码（如 API 路由），建议手动启动 Uvicorn 并开启热重载：

```
uvicorn.run(app, host='0.0.0.0', port=8080, reload=True, reload_dirs=['./'])
```

4. 生产环境部署

`ui.run_with(app)` 是生产环境部署的推荐方式，需结合 Gunicorn+Uvicorn 实现多进程部署：

```
# main.py (应用代码)
from nicegui import ui
from fastapi import FastAPI

app = FastAPI()
ui.run_with(app, mount_path='/ui')
ui.label('生产环境部署示例')

# 启动命令 (终端)
# gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker -b 0.0.0.0:8080
```

参数说明：

- `-w 4`：启动 4 个工作进程；
- `-k UvicornWorker`：使用 Uvicorn 工作器；
- `-b`：绑定地址和端口。

5. 中间件共享与优先级

自定义应用的中间件会作用于 NiceGUI 页面，优先级：

1. 自定义应用的全局中间件；
2. NiceGUI 内置中间件（如静态资源、WebSocket）；
3. 页面级中间件。

示例：为所有请求（包括 NiceGUI）添加日志中间件：

```
from fastapi import FastAPI, Request
from nicegui import ui

app = FastAPI()

# 全局日志中间件
@app.middleware('http')
async def log_middleware(request: Request, call_next):
    print(f'请求路径: {request.url.path}')
    response = await call_next(request)
```

```

return response

# 挂载NiceGUI，共享日志中间件
ui.run_with(app)
ui.label('带日志中间件的NiceGUI页面')

if __name__ == '__main__':
    import uvicorn
    uvicorn.run(app, host='0.0.0.0', port=8080)

```

6. 常见陷阱

- **应用实例重复挂载**：多次调用 `ui.run_with(app)` 会导致 NiceGUI 路由重复注册，引发 404/500 错误；
解决方案：确保 `ui.run_with(app)` 仅调用一次（如在应用初始化时）。
- **WebSocket 冲突**：NiceGUI 依赖 WebSocket 实现实时交互，若自定义应用禁用 WebSocket，会导致 UI 交互失效；
解决方案：确保 Uvicorn 启动时启用 WebSocket（默认启用），中间件不拦截 WebSocket 请求。
- **静态资源路径问题**：挂载到非根路径时，NiceGUI 内置静态资源（如 `_nicegui`）需确保路由正确；
解决方案：使用 `mount_path` 统一前缀，避免手动修改静态资源路径。
- **依赖注入失效**：NiceGUI 页面中无法直接使用 FastAPI 的 `Depends`，需通过 `ui.request` 手动调用：

```

from fastapi import Depends
async def get_user():
    return {'name': 'admin'}

# NiceGUI页面中调用依赖
async def show_user():
    user = await get_user() # 直接调用依赖函数
    ui.notify(f'当前用户: {user["name"]}')

ui.button('获取用户', on_click=show_user)

```

四、实战场景示例（完整集成 FastAPI）

```

from nicegui import ui, app as nicegui_app
from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from pydantic import BaseModel
import uvicorn

# 1. 定义数据模型
class Item(BaseModel):
    name: str
    price: float

# 2. 创建FastAPI应用
app = FastAPI(

```

```

    title='NiceGUI+FastAPI集成示例',
    description='API+可视化界面一体化服务',
    version='1.0'
)

# 3. 配置认证（示例：OAuth2）
oauth2_scheme = OAuth2PasswordBearer(tokenUrl='token')
def get_current_user(token: str = Depends(oauth2_scheme)):
    if token != 'valid-token':
        raise HTTPException(status_code=401, detail='认证失败')
    return {'username': 'admin'}

# 4. API路由（带认证）
@app.get('/api/items')
async def read_items(user: dict = Depends(get_current_user)):
    return [{'name': '商品1', 'price': 99.9}, {'name': '商品2', 'price': 199.9}]

@app.post('/api/items')
async def create_item(item: Item, user: dict = Depends(get_current_user)):
    return {'message': '创建成功', 'item': item}

# 5. 挂载NiceGUI到/ui路径，共享应用配置
ui.run_with(app, mount_path='/ui', title='商品管理界面')

# 6. NiceGUI可视化界面（调用API）
async def load_items():
    try:
        # 调用API，携带认证Token
        response = await ui.request.get(
            '/api/items',
            headers={'Authorization': 'Bearer valid-token'}
        )
        items = response.json()
        # 渲染商品列表
        for item in items:
            ui.card(f'名称: {item["name"]} | 价格: ¥{item["price"]}').classes('w-full my-2')
    except Exception as e:
        ui.notify(f'加载失败: {e}', type='error')

# NiceGUI UI布局
ui.label('商品管理可视化界面').classes('text-2xl font-bold my-5')
ui.button('加载商品列表', on_click=load_items)
ui.separator()
item_list = ui.column() # 商品列表容器

# 7. 启动应用（生产环境建议用Gunicorn）
if __name__ == '__main__':
    uvicorn.run(
        app,
        host='0.0.0.0',
        port=8080,
        reload=True, # 开发模式热重载
        reload_dirs=['./'] # 监控当前目录文件变化
    )

```


